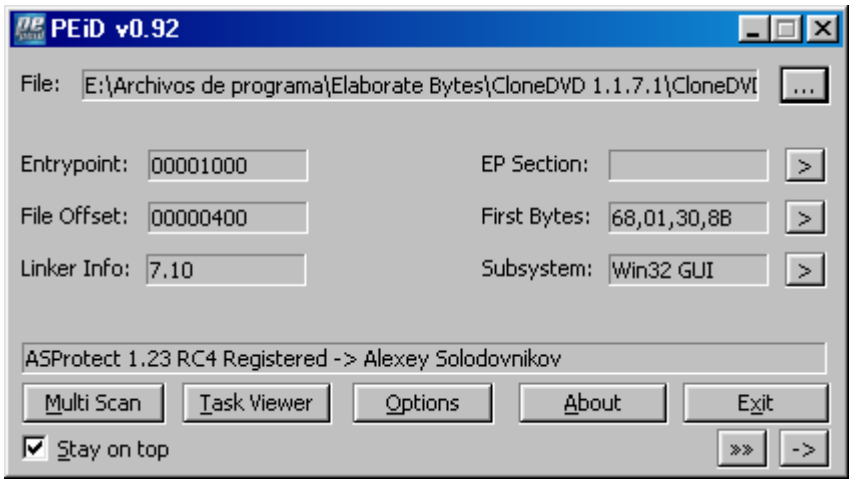
Programa:	CloneDVD 1.1.7.1	
Protección:	ASProtect + Trial 30 días	
Objetivos:	Desempaquetar el archivo y evitar el Trial.	
URL descarga:	---	
Dificultad:	Fácil si tienes Stripper, jeje… ;DDD	
Herramientas:	SoftIce + PEiD + Stripper 2.07f + UltraEdit32.	
Cracker:	SyXe'05 [hAcK-c0d3d]	Fecha: 17-07-2004

Aquí estamos hoy con otro programilla de esos utilizador para grabar DVD's, últimamente está el mercado inundado de éstos, pues bien nuestro paciente es una versión totalmente funcional de CloneDVD pero con una limitación Trial de 30 días, pues cada vez que ejecutamos el programa nos sale un mensajito diciendo los días que nos restan por 'consumir' y cuando gastemos todos saldrá otro mensajito pero diciendo que nuestro tiempo de prueba ha finalizado o algo así, bien imagino que estamos ante el típico esquema de protección para estos casos, lo haremos más fácil: como a le he echado un vistazo al tema os explicaré dos formas de crackear el programa ;-)

Primera parte: **Eliminando la limitación temporal...**

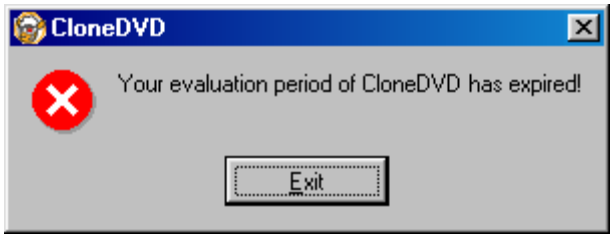
Bien, lo primero que haremos será desensamblar el ejecutable CloneDVD.exe con el wdasm por ejemplo ¿pero qué ocurre? Lo intentamos pero no lo conseguimos, si miramos con el PEiD vemos que está empacado:



Lo que tenemos que hacer antes de continuar es descomprimirlo, pero necesitamos el descompresor específico para ASProtect 1.23 RC4, yo lo he hecho con una gran utilidad llamada Stripper v2.07f que bajé desde la red hace algún tiempo de alguna página que visité, no recuerdo si fue en la e Karpoff, o la de Welcome.to.Craaaack o alguna de este tipo, pero en fin que por alguna página de estas lo tenéis, pues con este programita que se carga los AsPack y los AsProtect que da gusto recuperamos una copia del CloneDVD ya desempacada ;-)

Es de recalcar que si no tuviéramos esta gran utilidad podríamos hacerlo manualmente con ayuda del SICE y de algún volcador de procesos pero no todo sería tan fácil porque AsProtect jode la IAT (tabla de Importaciones) del ejecutable de manera que no funcionaría, para ello utilizaríamos Revirgin, Import Reconstructor y un poco de tiempo para repararla y aún así nos dejaría algunas entradas irreparadas que deberíamos hacer manualmente, como veis Stripper 2.07f nos ha ahorrado bastante tiempo ¡y dolores de cabeza! Hay quién se divierte más haciéndolo manualmente sino leed tutos de AuspeX, Ricardo Narvaja, etc., que son extraordinarios.

Y dicho esto seguimos con lo nuestro, como ya tenemos el ***.exe*** descomprimido podremos desensamblarlo tranquilamente con wdasm, una vez desensamblado pulsamos el botón Str.Refs (Referencias a Cadenas), nos mostrará una lista de todas las cadenas del programa y entre ellas está la del mensaje de error, en este caso es éste:



Pues buscamos 'Your evaluation period of CloneDVD has expired!' cuando la encontréis haced doble clic sobre ella para caer en la zona de código en la que se pinta el mensaje en pantalla y mirad alrededor, seguramente encontraréis un salto condicional que evite el mensaje o bien una referencia a una dirección desde donde se llega a éste, para este caso concreto tenemos los siguiente:

```
:0040339D 3975BC          cmp dword ptr [ebp-44], esi
:004033A0 0F8588050000    jne 0040392E          ;salto que evita los dos mensajitos nag!
:004033A6 3975A0          cmp dword ptr [ebp-60], esi
    • • •                  • • •
:004033B0 0F85A2030000    jne 00403758          ;salto hacia mensaje evaluation!
:004033B6 8D4DE8          lea ecx, dword ptr [ebp-18]
:004033B9 E822BF0700      call 0047F2E0
    • • •                  • • •
:004033E4 83C40C          add esp, 0000000C
:004033E7 8BF8            mov edi, eax
:004033E9 56              push esi
:004033EA 8D4DD0          lea ecx, dword ptr [ebp-30]

* Possible StringData Ref from Data Obj ->"Your evaluation period of %1 has "
                                        ->"expired!"
:004033ED 687CEF6A00      push 006AEF7C         ;aquí el mensaje_expired a la pila!!
:004033F2 51              push ecx
:004033F3 C645FC1C        mov [ebp-04], 1C
    • • •                  • • •
* Referenced by a (U)nconditional or (C)onditional Jump at Address:
:004033B0(C)    ;aquí llega el salto anterior!!
:00403758 8D4DDC          lea ecx, dword ptr [ebp-24]
:0040375B E880BB0700      call 0047F2E0
    • • •                  • • •
:00403798 83C40C          add esp, 0000000C
:0040379B 8BF8            mov edi, eax
:0040379D 56              push esi
:0040379E 8D55C8          lea edx, dword ptr [ebp-38]

* Possible StringData Ref from Data Obj ->"%1 is not registered, you have "
:004037A1 68E8EE6A00      push 006AEEE8    ;mensaje_evaluation a la pila!
:004037A6 52              push edx
:004037A7 C645FC22        mov [ebp-04], 22
```

Tenemos ante nosotros la zona que controla todo el meollo de si estamos registrados o no, y en caso de no estarlo si nos quedan días de prueba o no, para resumir un poco:

1.- Si saltamos en :004033A0 es que estamos registrados.
2.- Si saltamos en :004033B0 es que no estamos registrados pero nos quedan días de evaluación...
3.- si no salta en ninguno de ellos es que ni estamos registrados ni nos quedan días de evaluación **;(**

La solución está bastante clara, lo que debemos hacer es forzar el salto en :004033A0 para evitar las comprobaciones y acceder directamente al programa aún habiendo expirado el periodo de evaluación **;D**

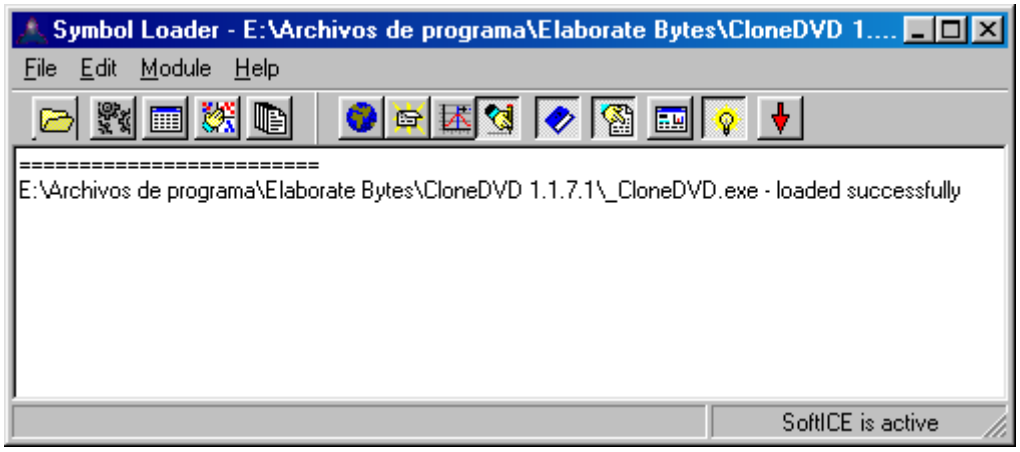
Ahora bien, si hacemos esto accederemos sin problema al programa como dije antes pero no estaremos registrados, para confirmarlo mirar en la ventanita About veréis esto:

Se ve claramente el Unregistered que indica que somos usuarios no registrados, o sea, tenemos el programa medio crackeado desde mi modo de ver ya que no estamos registrados, evitaremos esto en la segunda parte de este tutorial.

Segunda parte: **Convertirnos en usuarios Registrados...**

Bien, podemos trabajar o bien con una copia del original o bien con el crack anterior ya que los cambios que vamos a realizar son totalmente compatibles con el crack de antes, entremos en materia:

Para atacar lo que haremos será cargar el programa en el symbol loader del SoftIce si no sabéis que es ¿qué hacéis leyendo este tutorial? vale que está orientado a los que menos saben pero tampoco es para tanto, para esos fines existen infinidad de cursillos en la Red y aparte tenéis el manual de ayuda del SoftIce en el que explica de la A a la Z todos sus contenidos, etc., pero bueno por hoy seré benévolo y os lo explicaré (que no sirva de precedente que conste en acta, jeje) bien, el Symbol Loader es una pequeña utilidad que acompaña al SICE (SoftIce) y que nos servirá para "decirle" a éste qué programa queremos depurar (tracear), lo que haremos será ya en el sloader pulsar File/Open y seleccionar el programa en cuestión, bueno su ejecutable para ser más precisos, de esta manera se cargará en el propio sloader, pero para cargarlo en el SICE lo que haremos será pulsar sobre el botón de las ruedecillas dentadas que hay en la parte superior izquierda, mira os pongo la foto y acabo antes...



Aquí lo tenéis ¿veis el botón ahora? Además fijaos que tenemos cargado el CloneDVD como se puede ver en la bandeja, bien, pulsamos el botón citado para que salte el SICE, en este momento estaremos en la primera instrucción del programa, a partir de aquí iremos traceando con F10 hasta que aparezca el mensaje en pantalla, pero antes adelantaremos el reloj del windowx un par de meses para expirar el programa:

Tras este rollo seguimos, queremos registrarnos ¿verdad? Pues lo que tendremos que hacer es hacerle saber al programa que somos usuarios registrados, en primer lugar lo que haremos será echar un vistazo al código desensamblado desde wdasm (listado muerto que se le llama), miramos entre sus cadenas (strings) con el botón Strn Ref que ya conocemos, se desplegará un listado con todas las frases del programa y entre ellas vemos esta:

Serial ==> muy llamativa ¿verdad?

Hacemos doble clic encima para caer de pleno en la siguiente zona:

```
* Reference To: advapi32.RegOpenKeyExA, Ord:0000h
:005ED1BC FF1510E06A00      Call dword ptr [006AE010]     ;busca la clave del registro siguiente:
:005ED1C2 85C0              test eax, eax                 ;HK_LM\Software\Elaborate Bytes\CloneDVD\CDKey
:005ED1C4 7574              jne 005ED23A                  ;si la encuentra no salta…
:005ED1C6 8B4C2408          mov ecx, dword ptr [esp+08]
    • • •                    • • •
* Possible StringData Ref from Data Obj ->"Serial"
:005ED1D6 6890857800        push 00788590
:005ED1DB 51                push ecx
:005ED1DC C744241C50000000  mov [esp+1C], 00000050

* Reference To: advapi32.RegQueryValueExA, Ord:0000h
:005ED1E4 FF151CE06A00      Call dword ptr [006AE01C]     ;aquí busca el valor serial
:005ED1EA 85C0              test eax, eax                 ;existe este valor en el registro?
:005ED1EC 7541              jne 005ED22F                  ;si es así  no saltará...
```

```
:005ED1EE 57                push edi                ;continuamos normalmente...
```

Hemos localizado el momento exacto en que accede al registro en búsqueda de la clave Serial dentro de:

HKEY_LOCAL_MACHINE\Software\Elaborate bytes\CloneDVD\CDKey

Que previamente deberá existir claro está pues de no ser así saltaríamos directamente en ***:005ED1C4*** y no llegaría a buscar este valor, hasta aquí todo perfecto pero ¿ahora qué hacemos? Fácil, será continuar a ver con quién compara el serial que allí vamos a poner ya que no existe. Entonces creamos la clave ***CDKey*** que antes no existía en el lugar indicado (ver arriba) y dentro de ésta el Valor de Cadena **Serial** que rellenaremos con cualquier chorrada. Lo que estamos haciendo es engañar al programa para que encuentre los datos que busca cuando acceda al registro, bien ahora lanzamos el programa desde el symbol loader pero antes ponemos un ***BPX 5ED1BC*** para que para allí puesto que queremos ver qué hace con nuestro serial...

Si seguimos estos pasos SICE romperá en esa dirección, traceamos con F10 hasta llegar a :005ED1EE veréis que los dos saltos previos ahora no se han producido (antes sí), esto es debido a que ahora el programa sí encuentra en el registro lo que andaba buscando, pues seguiremos traceando porque ahora deberá verificar el serial para saber si es o no correcto. Perfecto, es cuestión de ir pulsando F10 hasta que ocurra algo, ese algo será ver el mensaje de 'Program Expired' que no nos deja entrar en el programa, como sabemos dónde lo pinta tenemos que las comprobaciones del serial se encontrarán entre el lugar en que éste accede al registro :005ED1EA y dicho mensaje :004033F7, pues entre estas direcciones debe tomar la decisión de sacar el mensaje o continuar, esto es lógico de pensar ya que así sería si estuviéramos verdaderamente registrados pues entraríamos en el programa sin ver el mensaje.

Cuando recoge el serial si os fijáis poco después hay un ret (retorno de función) en :005ED256 que nos envía a:

```
:005ED634 E867FBFFFF        call 005ED1A0           ;recogemos el serial!
:005ED639 83C408            add esp, 00000008       ;salimos aquí
:005ED63C 85C0              test eax, eax           ;es eax=0? Si es así es que la clave buscada no existe!
:005ED63E 742D              je 005ED66D             ;si es así entonces saltamos => salto_MALO
:005ED640 8B8C2464010000    mov ecx, dword ptr [esp+00000164]
    •  •  •                 •  •  •
:005ED647 51                push ecx                ;si existe la clave seguimos ;-)
:005ED648 6A00              push 00000000
:005ED64A 8D44240C          lea eax, dword ptr [esp+0C]
:005ED64E E80DFCFFFF        call 005ED260           ;aquí verifica el serial!!!
:005ED653 83C408            add esp, 00000008
:005ED656 85C0              test eax, eax           ;es eax=0? Si lo es quiere decir que no es válido ;(
:005ED658 7413              je 005ED66D             ;y saltará consecuentemente
:005ED65A 56                push esi                ;y si es válido seguimos ;-D
:005ED65B 8D4C2458          lea ecx, dword ptr [esp+58]
:005ED65F C70602000000      mov dword ptr [esi], 00000002
:005ED665 E8E6FCFFFF        call 005ED350           ;aquí no llegamos si el serial no existe o si no es válido!
:005ED66A 83C404            add esp, 00000004

* Referenced by a (U)nconditional or (C)onditional Jump at Addresses:
|:005ED63E(C), :005ED658(C)
|
:005ED66D 8BC6              mov eax, esi

* Referenced by a (U)nconditional or (C)onditional Jump at Address:
|:005ED5CA(C)
|
:005ED66F 5E                pop esi
:005ED670 81C454010000      add esp, 00000154
:005ED676 C3                ret                     ;retornamos...
```

Bien ahora lo que haremos será buscar la forma de hacer pasar el serial falso por serial válido, si nos fijamos veremos que salimos del call 005ED260 con eax=0 lo que nos hace saltar en :005ED268 evitando que se ejecute un trozo de código ¿verdad? Pues si entramos en dicho call vemos que lo que hace es verificar el serial ¡siiiiii! Hemos pillado el call responsable de las comprobaciones, jeje reiniciamos el proceso y lo lanzamos desde el symbol loader pero ponemos un BPX 5ED64E para parar allí, lo lanzamos, saltará el SICE pulsamos F5 para dejar correr el programa y al poco vuelve a saltar el SoftIce ahora justo sobre la instrucción del call, jeje, entramos con F8 y observamos como rápidamente nos quiere sacar de éste con eax=0 ¡mal_ROLLO! Si evitamos este salto por ejemplo invirtiendo el flag zero con el comando **r fl z** y continuamos vemos que poco después entramos en otro call que devuelve eax=1 que no nos sirve porque después hay un salto que no se efectúa precisamente si eax=1 con lo que llegamos a la salida del call con eax=0 otra vez, tampoco nos interesa, si forzamos el salto modificando el flag zero otra vez entonces sí saltará para llegar a otra zona en la que sigue realizando comprobaciones, al final del todo hay una salida que modifica **al**

(al = parte baja de eax) asignándole un valor que será definitivo ya que justo después está el retorno, pondré un trozo de la rutina para que sirva de apoyo a lo explicado aquí:

```
* Referenced by a CALL at Address:
|:005ED64E
|
:005ED260 83EC64          sub esp, 00000064
:005ED263 57              push edi
:005ED264 8BF8            mov edi, eax
:005ED266 C7442408FFFFFFFF mov [esp+08], FFFFFFFF
:005ED26E E88DFEFFFF      call 005ED100   ;realiza comprobaciones!
:005ED273 85C0            test eax, eax   ;es eax=0?
:005ED275 742A            je 005ED2A1     ;si lo es saltará => salto_MALO!
    • • •                 • • •
:005ED28D 51              push ecx        ;si no seguimos…

* Possible StringData Ref from Data Obj ->"%u%c%u%c%u"
                    |
:005ED28E 6884857800      push 00788584
:005ED293 57              push edi
:005ED294 E8B5DE0600      call 0065B14E         ;más comprobaciones!
:005ED299 83C41C          add esp, 0000001C     ;reajusta la pila
:005ED29C 83F805          cmp eax, 00000005     ;es eax=5?
:005ED29F 7407            je 005ED2A8           ;si es así seguimos => salto_BUENO!

* Referenced by a (U)nconditional or (C)onditional Jump at Address:
|:005ED275(C)
|
:005ED2A1 33C0            xor eax, eax          ;hace eax:=0 => Algo ha ido MAL!!!
:005ED2A3 5F              pop edi
:005ED2A4 83C464          add esp, 00000064
:005ED2A7 C3              ret                   ;y nos saca del call


* Referenced by a (U)nconditional or (C)onditional Jump at Address:
|:005ED29F(C)   ;referenciado desde aquí!
|
:005ED2A8 8B442404        mov eax, dword ptr [esp+04]     ;el salto anterior viene aquí!!
:005ED2AC 33442470        xor eax, dword ptr [esp+70]     ;seguimos comprobando...
:005ED2B0 50              push eax
:005ED2B1 8D54241C        lea edx, dword ptr [esp+1C]

* Possible StringData Ref from Data Obj ->"%u"
                    |
:005ED2B5 6898857800      push 00788598
:005ED2BA 52              push edx
    • • •                 • • •
:005ED31B E810FDFFFF      call 005ED030
:005ED320 8B4C242C        mov ecx, dword ptr [esp+2C]
:005ED324 8B542424        mov edx, dword ptr [esp+24]
:005ED328 83C418          add esp, 00000018
:005ED32B 33C0            xor eax, eax
:005ED32D 3BCA            cmp ecx, edx    ;comparación determinante, ecx y edx deben tener el mismo valor
:005ED32F 5E              pop esi
:005ED330 0F94C0          sete al         ;aquí al toma su valor que deberá ser 1
:005ED333 5F              pop edi
:005ED334 83C464          add esp, 00000064
:005ED337 C3              ret             ;retornamos…
```

Explicaré por encima cómo funciona esto que estamos viendo: En primer lugar decir que el primer salto en :005ED275 está controlado por el call previo el cual es accedido en dos ocasiones a lo largo del código, aquí podríamos crackear la rutina para que devolviera eax=1, bien si seguimos hacia abajo vemos otro salto que se produce si eax vale cinco pues podríamos crackear la rutina para que asignara ese valor a eax ya que es quién modifica este registro, y finalmente el último escollo a salvar es este:

```
:005ED32D 3BCA            cmp ecx, edx    ;comparación determinante
:005ED32F 5E              pop esi
:005ED330 0F94C0          sete al         ;si el flag zero está activado invertirá el valor de al
```

Decir que este trozo de código hace lo siguiente: primero compara los registros ecx y edx entre sí. Si son iguales se activará el flag zero (sino no) y dos líneas después tenemos el **sete al** que lo que hace es mirar el flag zero y en caso de estar activado asignará un 1 al valor de AL, es decir si suponemos que el flag zero está activado AL pasará a valer 1 después de esta instrucción ¿fácil verdad?

Pues bien, como teníamos que el call debería devolver eax=1 (NOTA: si **al** vale 1 ya serviría puesto que es la parte baja del registro eax) si esto no está muy claro miradlo en algún cursillo de ensamblador de esos que rulan por la red, dicho esto seguimos: íbamos diciendo que el call en el que nos encontramos debería retornar eax=1 ¿no? Hemos entrados en éste y visto que existen tres puntos clave:

Primero:

```
:005ED26E E88DFEFFFF        call 005ED100    ;debe retornar eax=1
:005ED273 85C0              test eax, eax
:005ED275 742A              je 005ED2A1
```

Segundo:

```
:005ED294 E8B5DE0600        call 0065B14E    ;debe retornar eax=0
:005ED299 83C41C            add esp, 0000001C
:005ED29C 83F805            cmp eax, 00000005
:005ED29F 7407              je 005ED2A8
```

Tercero:

```
:005ED32D 3BCA              cmp ecx, edx
:005ED32F 5E                pop esi
:005ED330 0F94C0            sete al          ;deberá valer 1
```

Pasemos a la acción, vamos a aplicar el parche, para el primer caso podemos cambiar (dentro del call):

```
:005ED10B 750A      jne 005ED117
:005ED10D 33C0      xor eax, eax

POR:
:005ED10B 33C0      xor eax, eax     ;ponemos eax a 0
:005ED10D 40        inc eax          ;incrementamos eax a 1
:005ED10E 90        nop              ;nopeamos el byte restante
```

Para el segundo caso cambiaremos el salto (dado que el call es enorme no lo tocaremos):

```
:005ED29F 7407      je 005ED2A8      POR:
:005ED29F EB07      jmp 005ED2A8
```

Y por último para el tercer caso haremos:

```
:005ED330 0F94C0    sete al

POR:
:005ED330 B001      mov al, 01       ;ponemos al a 1 como necesitamos!
:005ED332 90        nop              ;nopeamos el byte restante!
```

Aplicando estos últimos cambios y escribiendo en el registro los datos adecuados obtendremos nuestra propia versión de CloneDVD 1.1.7.1, ahora si miráis en About encontraréis vuestro nombre y compañía allí cosa que antes no pasaba si recordáis, con lo que podemos decir que habéis crackeado el programa, y lo que es mejor: de dos maneras distintas, acordaos que ya lo teníamos crackeado ya que podíamos utilizarlo ilimitadas veces, lejos de los 30 días de evaluación que nos permitía, pero de este último modo además de vencer el TimeTrial veremos nuestro nombre en la ventana About lo que indica que somos usuarios registrados jeje **;DD**

Bien, como dije antes deberemos escribir nosotros mismos los datos en el registro , para esto nos podríamos preparar un pequeño archivo con extensión .reg de manera que lo haga por nosotros, pensad que pasaría cada vez que reinstaléis el programa porque has formateado el disco o que lo instaléis en otra máquina ¿repetir el proceso del registro otra vez? Sería un poco pesado, de aquí nace la necesidad de crearnos el susodicho archivo, yo aquí os pondré

un ejemplo, cada uno se puede (o mejor dicho se debe) tomar la licencia de cambiar mi nombre por el suyo de modo que aparezca esté registrado para él, el archivo sería algo así:

----------// cortar por aquí //----------
REGEDIT4

[HKEY_LOCAL_MACHINE\Software\Elaborate Bytes\CloneDVD\CDKey]
"Serial"="12345-12345-12345-123"
"Name"="[SyXe]"
"Company"="hAcK-c0d3d"

----------// cortar por aquí //----------

Hacéis doble clic sobre él y solito se autoejecutará **;-)**

Para usuarios de Windows NT (Win2000, XP...) podréis hacerlo de esta manera:

----------// cortar por aquí //----------
Windows Registry Editor 5.00

[HKEY_LOCAL_MACHINE\Software\Elaborate Bytes\CloneDVD\CDKey]
"Serial"="12345-12345-12345-123"
"Name"="[SyXe]"
"Company"="hAcK-c0d3d"

----------// cortar por aquí //----------

Lo único que cambia es la cabecera, digamos la “manera de acceder” al registro, el resto es idéntico en todos lo Windowx (que yo sepa claro)

Y damos por terminado este eterno tutorial que espero no os haya dormido (lo entendería) pues no sé si era necesario tanto escrito, en fin a lo hecho..., sólo queda decir que espero que haya servido a alguien para lo que sea, con esa motivación lo he escrito, más que nada para aquellos que se inician y necesitan ver cosas fáciles como esta y sobretodo extensamente explicadas ya que toda ayuda es poca creo yo.

Pronto volveremos al ataque con nuevos programas y aventuras que nos harán pasar un buen rato **;-)**

Y con esto damos por finalizado el tuto, espero que os haya gustado, pronto volveremos con otros, si alguien quiere contactar conmigo por cualquier tema, ya sea una duda, una sugerencia o como no una crítica siéntase libre de escribirme a cualquiera de estas direcciones:

syxe00@yahoo.es
syxe00@hotmail.com